Introdução aos processadores programáveis

Henrique Bernardes

3 de novembro de 2025

1 Início - Arquitetura básica de um processador programável

Começamos aqui esclarecendo as diferenças entre hardware dedicado e um processador de propósito geral, tendo como objetivo diferenciar o que estávamos trabalhando do que vamos passar a trabalhar.

Um hardware dedicado implementa circuitos digitais para realizar uma determinada tarefa, podendo assim ser costumizado. Além disso, principalmente pela capacidade de paralelização, hardware dedicado tem um alto desempenho. Contudo, apresenta alto custo e pouca flexibilidade, uma vez que qualquer alteração reflete uma modificação no circuito.

Já um processador de propósito geral tem seu "programa"armazenado em memória, podendo ser modificado, uma vez que se baseia em software, acarretando, também, em um menor tempo de desenvolvimento. Contudo, pelo fato de aqui termos uma sequência de instruções ao invés de um fluxo de dados, é difícil paralelizar o algoritmo, apresentando um baixo desempenho. Dentre os diversos processadores de propósito geral podem ser listados os Intel, AMD, ARM, MIPS e ATMega.

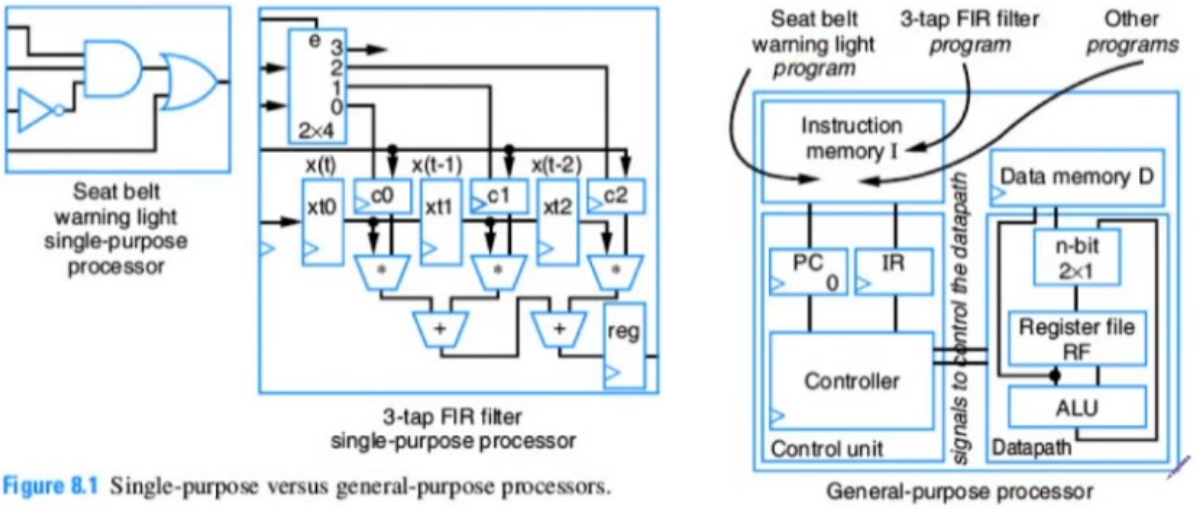

Figura 1: Hardware dedicado versus processador de propósito geral

2 Arquitetura load-store

2.0.1 Caminho de dados

A memória de dados armazena todos os dados que o processador pode acessar. Esses dados devem passar pelo banco de registradores antes de ser processados pela Unidade Lógica Aritmética(ULA), armazenando os dados a serem operados pela ULA. Ao longo do caminho, o multiplexaddor 2x1 escolhe entre os dados da memória ou os resultados intermediários da ULA.

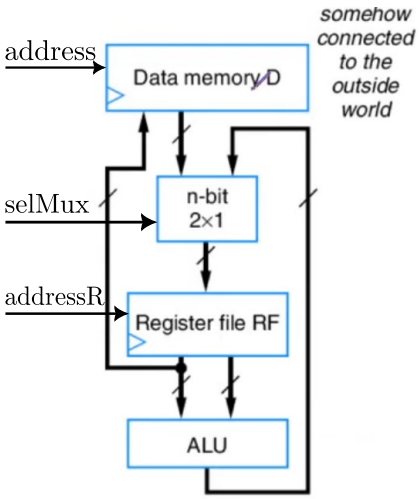

Figura 2: Caminho de dados de uma arquitetura load-store.

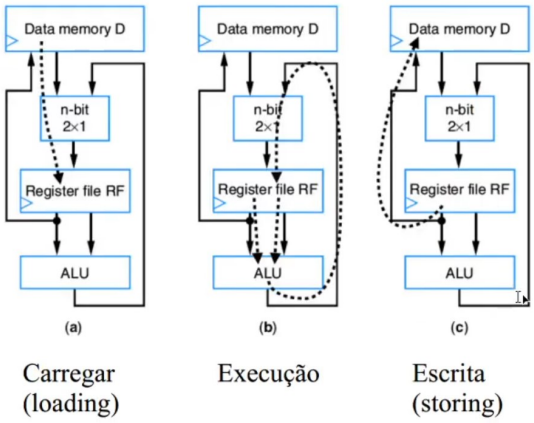

Figura 3: Casos de carregação, execução e escrita de dados em uma arquitetura load-store

Ou seja, o processamento ocorre por meio destas três instruções (figura 3):

1. Carregar dados - leitura de um determinado *address* e carregamento no banco de registradores;
2. Transformar dados - executar operações;
3. Armazenar dados - escrita - um resultado intermediário vindo da ULA pode ser enviada de volta para a memória de dados (também, para o endereço *address*)

2.0.2 Unidade de Controle

A unidade de controle está para o caminho de dados como uma FSM, de fato, como é de se esperar, ela controla o caminho de dados; ou seja, nesta unidade são configurados os sinais de *clear*, *load*, *addresses* da memória de dados e do banco de registradores e seleção do multiplexador.

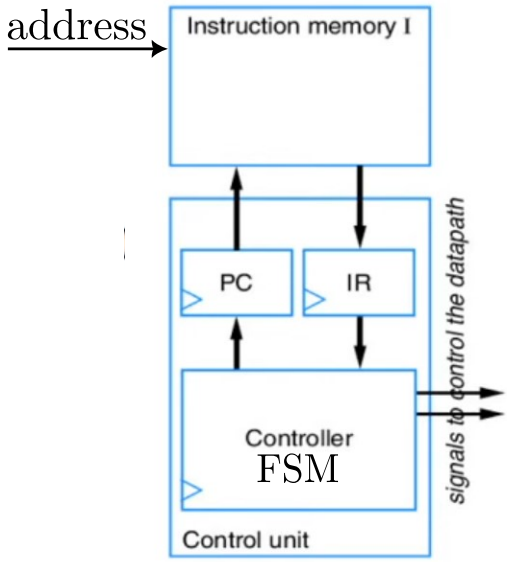

Figura 4: Unidade de controle básica.

Inicialmente, a unidade lê as instruções da memória de instruções I.

Em uma arquitetura FDx de três estágios, as intruções são realizadas em uma média total de três ciclos de clock, baseadas em três componentes:

- PC (*program counter*): Contador que indica a próxima instrução a ser lida;
- IR (*instruction register*): Armazena a instrução;
- Controlador(*Controller*): FSM

As operações são descritas abaixo:

1. Busca (***F****etch*): Lê I[address] e armazena no registrador local IR(*instruction register*);
2. Decodificação (***D****ecode*): Determina a operação a ser executada;
3. Execução (*E****x****ecute*): Gera os sinais de controle para o caminho de dados de modo que a instrução seja executada.

A figura 5 demonstra a interface entre o *controller*(FSM) e o resto da unidade:

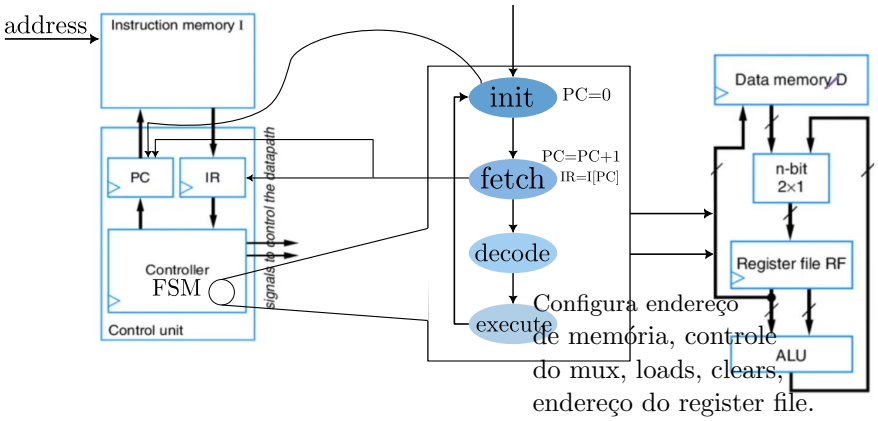

Figura 5: Interface entre os elementos na unidade de controle e a FSM em si.

Temos como exemplo a seguinte operação a ser realizada: $D[9] = D[0] + D[1]$

```
RF[0]=D[0]          – load
RF[1]=D[1]          – load
RF[2]=RF[0]+RF[1]   – (ULA)
D[9]=RF[2]          – write
```

→ Aqui, o valor na posição 2 do register file é escrito na posição 9 do data memory.

Na figura 7 observa-se a primeira linha se propagrando ao longo da unidade de controle e caminho de dados

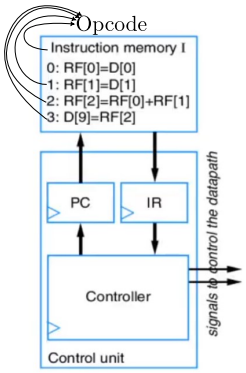

Figura 6: Instruções e armazenamento no *instruction memory I*.

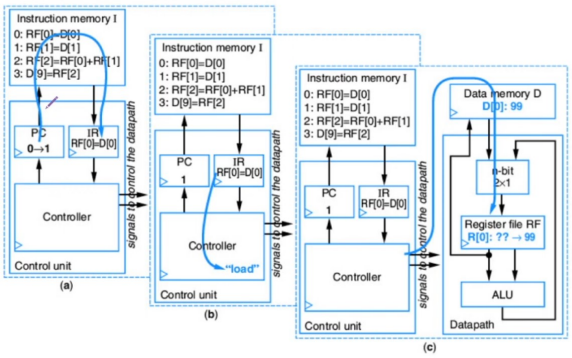

Figura 7: Primeira linha de código se propagando pela unidade de controle e caminho de dados.

3 Processador programável de três instruções

Inicialmente, definimos o que é conhecido como **conjunto de instruções** do processador programável: é uma lista das instruções possíveis e a meneira de representar essas instruções na memória. Vamos definir, também, que um processador usa instruções de 16 bits e que a memória de instruções I tem 16 bits de largura. Normalmente, um determinado número de bits é reservado pelo conjunto de instruções para indicar a instrução em si a ser realizada sendo que os restantes especificam informações adicionais como registradores de origem, destino, etc.

Definiremos um conjunto de três operações símples, onde os 4 bits mais significativos irão representar a operação e os 12 restantes indicam os endereços no banco de registradores(*register file*) e na memória de dados.

Teremos, então, as seguintes instruções:

```
(0) Carregar - load - 0000(r3)(r2)(r1)(r0)(d7)(d6)(d5)(d4)(d3)(d2)(d1)(d0)
(1) 0000 0000 00000000
(2) 0000 0001 00101010
```

Esta instrução especifica uma movimentação de dados que vai de uma posição da memória de dados(especificado pelos d_i bits) até um registrador no banco de registradores(especificado pelos r_i bits), ou seja, funciona como uma instrução de *fetch*.
Como exemplo, primeiro, a linha 1 indica uma operação de movimentação de dados do da posição 0 da memória de dados($D[0]$) até a posição 0 do banco de registradores($RF[0]$), ou seja, esta instrução realiza a operação $RF[0] = D[0]$.
A linha 2 indica uma operação de movimentação de dados da posição 42 da memória de dados ($D[42]$) até a posição 1 do banco de registradores ($RF[1]$). Ou seja, especifica a operação $RF[1] = D[42]$

```
(0) Armazenar - store -0001(r3)(r2)(r1)(r0)(d7)(d6)(d5)(d4)(d3)(d2)(d1)(d0)
(1) 0001 0000 00001001
```

Essa instrução especifica uma movimentação de dados no sentido oposto da instrução anterior, indo do banco de registradores até a memória de dados, ou seja, funciona como uma instrução de *store*.
Assim sendo, a instrução da linha 1 armazena no endereço 9 da memória de dados ($D[9]$) o dado que está localizado no endereço 0 do banco de registradores($RF[0]$).

```
(0) Somar-add-0010(ra3)(ra2)(ra1)(ra0)(rb3)(rb2)(rb1)(rb0)(rc3)(rc2)(rc1)(rc0)
(1) 0010 0010 0000 0001
```

Essa instrução especifica somar os conteúdos de dois registradores do banco de registradores, especificados por $rb_3rb_2rb_1rb_0$ e $rc_3rc_2rc_1rc_0$ e então armazenar no registrador do banco de registradores especificado por $ra_3ra_2ra_1ra_0$.
Assim sendo, a instrução da linha 1 especifica somar o conteúdo do registrador $RF[0]$ ao conteúdo do registrador $RF[1]$ e armazenar no registrador $RF[2]$, ou seja, $RF[2] = RF[0] + RF[1]$.

Nenhuma dessas operações modifica os conteúdos dos operandos de origem, ou seja:

- A instrução **carregar** faz com que os conteúdos de um endereço da memória de dados $D[i]$ sejam carregados em um registrador do banco de registrador $RF[i]$, sem alterar o conteúdo naquele endereço da memória de dados $D[i]$.

- A instrução **armazenar** faz com que os conteúdos de um endereço do banco de registrador $RF[i]$ sejam armazenados em um endereço da memória de dados $D[i]$ sem alterar o conteúdo daquele registrador $RF[i]$.
- A instrução **somar** faz com que os dados dos registradores $Rb[i]$ e $Rc[i]$ sejam somados e armazenados em $Ra[i]$ sem alterar os conteúdos dos registradores $Rb[i]$ e $Rc[i]$.

Assim sendo, é possível entender o processo para realizar a operação $D[9] = D[0] + D[1]$ (figura 8):

```
OPERAÇÃO D[9]=D[0]+D[1]

    (0) load - RF[0] = D[0] - 0000 0000 00000000
    (1) load - RF[1] = D[1] - 0000 0001 00000001
    (2) add - RF[2]=RF[0]+RF[1] - 0010 0010 0000 0001
    (3) store - D[9] = RF[2] - 0001 0010 00001001
```

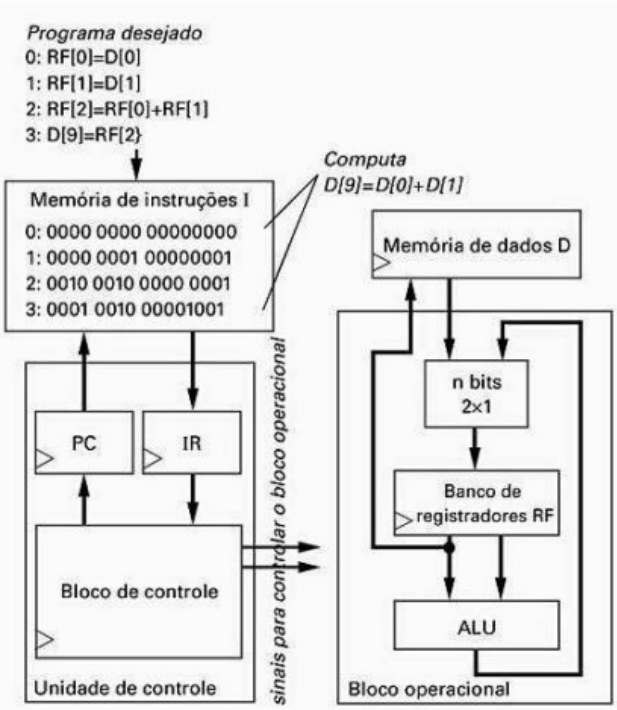

Figura 8: $D[9] = D[0] + D[1]$

Realizando a operação $D[5] = D[5] + D[6] + D[7]$:

```
OPERAÇÃO D[5] = D[5] + D[6] + D[7]:

(0) 0000 0000 00000101   // load  -  RF[0] = D[5]
(1) 0000 0001 00000110   // load  -  RF[1] = D[6]
(2) 0000 0010 00000111   // load  -  RF[2] = D[7]
(3) 0010 0011 0000 0001  // sum   -  RF[3] = RF[0]+RF[1] ou D[5]+D[6]
(4) 0010 0011 0011 0010  // sum   -  RF[3] = RF[3]+RF[2] ou D[5]+D[6]+D[7]
(5) 0001 0011 00000101   // store -  D[5]  = RF[3]

otimizando para utilizar apenas 3 registradores do banco de registradores

(0) 0000 0000 00000101   // load  -  RF[0] = D[5]
(1) 0000 0001 00000110   // load  -  RF[1] = D[6]
(2) 0000 0010 00000111   // load  -  RF[2] = D[7]
(3) 0010 0011 0000 0001  // sum   -  RF[0] = RF[0]+RF[1] ou D[5]+D[6]
(4) 0010 0011 0011 0010  // sum   -  RF[0] = RF[0]+RF[2] ou D[5]+D[6]+D[7]
(5) 0001 0011 00000101   // store -  D[5]  = RF[0]
```

Finalmente, como foi visto, na memórias de instruções, as instruções existem como sequências de 0s e 1s, representação denominada como **código de máquina**. Como é de se esperar, escrever um programa inteiro dessa forma acarretaria em muitos erros. Por esse motivo, utiliza-se uma ferramenta chamada *assembler*, que permite escrever instruções usando mnemônicos, ou símbolos, que o *assembler* traduz para código máquina. Um código escrito em mnemônicos é chamado de **código assembly**. Dessa forma, no contexto do conjunto de três instruções, um *assembler* nos permite escrever as instruções usando os seguintes mnemônicos:

1. Carregar - MOV Ra, d - especifica a operação $RF[a] = D[d]$. O valor de a pode assumir quaisquer valor de 0 a $2^4 - 1$, ou seja, 0 a 15, assim, R0 significa $RF[0]$, R1 significa $RF[1]$, etc. O valor de d pode assumir quaisquer valor de 0 a $2^8 - 1$, ou seja, 0 à 255.
2. Armazenar - MOV d, Ra - especifica a operação $D[d] = RF[a]$;
3. Somar - ADD Ra, Rb, Rc - especifica a operação $RF[a] = RF[b] + RF[c]$

Com o uso desses mnemônicos o programa $D[9] = D[0] + D[1]$ pode ser rescrito como:

```
OPERAÇÃO D[9]=D[0]+D[1]:

(0) MOV R0, 0
(1) MOV R1, 1
(2) ADD R0, R0, R1
(3) MOV 9, R0
```

Tendo estabelecido a base dos processadores de três instruções, como suas instruções e a arquitetura básica, podemos expandir o bloco fsm da figura 5, principalmente, o estado *execute*.

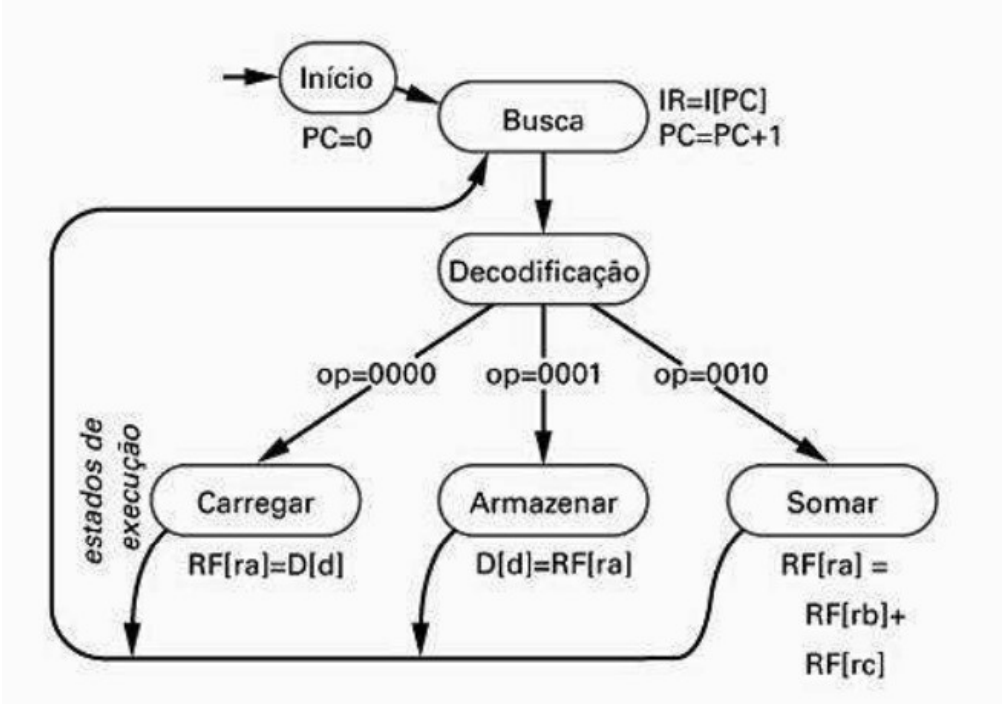

Figura 9: FSM com estado execute expandido.

Primeiramente, assumimos que op é uma abreviação de $IR[15...12]$, ou seja, os bits mais significativos do *instruction register*. O mesmo pode ser dito sobre ra($IR[11...8]$), rb($IR[7...4]$), rc($IR[3...0]$) e d($IR[7...0]$), ou seja:

- op $\rightarrow IR[15...12]$
- ra $\rightarrow IR[11...8]$
- rb $\rightarrow IR[7...4]$
- rc $\rightarrow IR[3...0]$
- d $\rightarrow IR[7...0]$

Feito isso, passamos para o bloco de operacional, agora, mostrando com mais detalhes a integração entre ele e a unidade de controle:

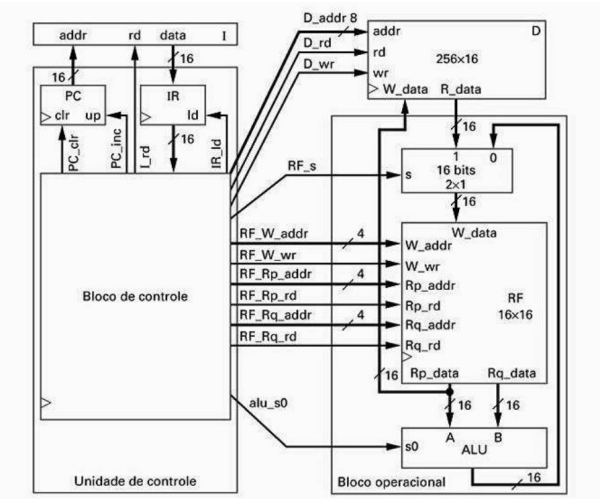

Figura 10: Unidade de controle e bloco operacional e interfaces entra ambos para um processador de três instruções.

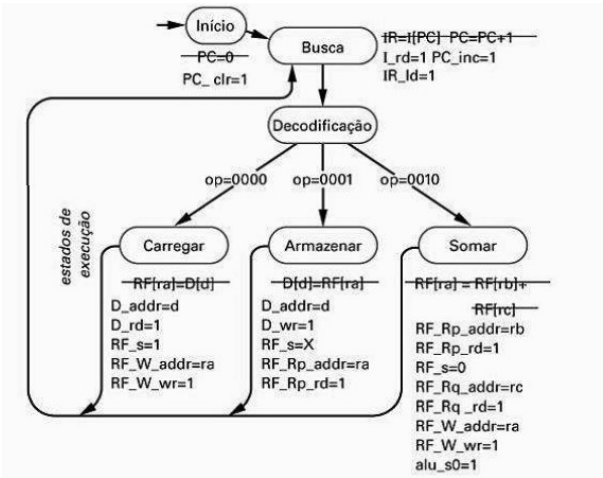

Figura 11: FSM de baixo nível do processador de três instruções.

Por cada instrução possuir 4 bits para endereçar o banco de registradores, ele dispõe de 16 registradores. Também, o bloco operacional tem um sinal de controle para a ALU(ULA) chamado alu_s0. Quando alu_s0=1, a ULA soma as duas entradas; quando alu_s0=0, ela simplesmente deixa passar a entrada. A memória de dados tem 256 palavras, uma vez que cada instrução possui 8 bits para endereçar a memória de dados.

Finalmente, podemos obter a FSM de baixo nível, substituindo as ações de alto nível pelas operações booleanas de saída, vista na figura 11.

Analisando o comportamento da FSM podemos ver como um programa seria executado nesta arquitetura de processador de três instruções:

1. A FSM naturalmente começa no estado Início, com $PC_clr = 1$, limpando o *program counter* deixando-o em 0;

2. No próximo ciclo de clock, a FSM entra no estado de busca, no qual lê a memória de instruções no endereço 0(porque $PC = 0$) e carrega o valor lido em IR. Ao fim, incrementa o valor de PC. *Algo como* $IR[PC] = IM[PC]$;

3. Em seguida, a FSM entra no estado de decodificação, sem ações realizadas, estando aqui para permitir que os valores dos códigos de operação se atualizem(as atualizações de registrador atribuídas a um estado só ocorrem de fato na próxima borda de clock). A depender dos 4 bits mais significativos do *instruction register*, o próximo estado será carregar, armazenar ou somar;

4. No estado de carregar, a FSM prepara as linhas de endereço de memória de dados utilizando os 8 bits menos significativos do *instruction register*)($D_addr = d$) e habilitando a leitura da memória de dados($D_rd = 1$). Aqui, a seletora do multiplexador é ativada($RF_s = 1$) para que o valor de saída da memória de dados vá para o banco de registradores. Finalmente, prepara o endereço de escrita do banco de registradores ($RF_W_addr = ra$), usando $IR[11...8]$ e habilitando a escrita ($RF_W_wr = 1$). Tudo isso faz com que o conteúdo que esteja na memória de dados seja carregado no registrador apropriado do banco de registradores.

5. De forma semelhante os estados Armazenar e Somar preparam as linhas de controle conforme a necessidade das operações de armazenamento e soma.

6. Por fim, a FSM retorna ao estado de Busca, buscando pela próxima instrução.

4 Processador programável de seis instruções

Tendo em vista as limitações de um processador de três instruções, estendemos o conjunto de instruções do processador acrescentando mais três instruções. Primeiramente, além das instruções vindas do processador de três instruções, *load*, *store* e *add*, uma nova instrução permitirá carregar uma constante para um dos registradores do banco de registradores, permitindo novas operações como $RF[0] = RF[1] + 5$, onde 5 é uma constante, ou seja,não se encontra na memória de dados.

Outra operação nova será a de subtração: similar à operação de soma já existente.

Finalmente, a sexta e última operação será a de "Saltar se zero". Esta instrução permitirá saltar para alguma parte do programa caso o conteúdo de algum registrador especificado for 0.

Expandindo nestas operações:

```
Carregar constante - 0011(r3)(r2)(r1)(r0)(c7)(c6)(c5)(c4)(c3)(c2)(c1)(c0)
```

Especifica que o número binário representado pelos bits c_i(conhecido como uma constante) deve ser deve ser carregado no registrador especificado pelos bits r_i. Aqui, r_i corresponde à $IR[11...8]$ e c_i corresponde à $IR[7...0]$. O mnemônico dessa instrução é:

```
MOV Ra, #c // RF[a]=c
```

onde a pode assumir qualquer valor de 0 até $2^4 - 1$ em complemento de 2, ou seja, de 0 até 15, e c pode assumir qualquer valor entre $\frac{-2^8}{2}$ e $\frac{2^8-1}{2}$, ou seja, entre -128 e $+127$.

```
Subtrair - 0100(ra3)(ra2)(ra1)(ra0)(rb3)(rb2)(rb1)(rb0)(rc3)(rc2)(rc1)(rc0)
```

Especifica a subtração dos conteúdos de dois registradores do banco de registradores especificados por rb_i e rc_i. O resultando é então armazenado no registrador do banco de registradores especificado por ra_i(Lembrando que ra corresponde $IR[11...8]$, rb corresponde $IR[7...4]$ e rc corresponde $IR[3...0]$). O mnemônico dessa instrução é

```
Sub Ra, Rb, Rc // RF[a]=RF[b]+RF[c]
```

Os termos a, b e c podem assumir qualquer valor de 0 até $2^4 - 1$ em complemento de 2, ou seja, de 0 até 15.

```
Saltar se zero - 0101(ra3)(ra2)(ra1)(ra0)(o7)(o6)(o5)(o4)(o3)(o2)(o1)(o0)
```

Especifica que se o conteúdo do registrador especificado por ra_i for 0, então carrega-se o PC com o valor atual de PC somado ao valor em oito bits e complemento de 2 o_i, podendo ser um *offset* positivo ou negativo. Aqui, ra_i corresponde à $IR[11...8]$ e o_i à $IR[7...0]$. O mnemônico dessa instrução é:

```
JMPZ Ra, offset // PC = PC + offset se RF[a]=0
```

Offset pode tomar qualquer valor entre $\frac{-2^8}{2}$ e $\frac{2^8-1}{2}$. Ou seja. podemos saltar para frente até 127 endereços ou para trás até 128 endereços. Aqui, também, onde a pode assumir qualquer valor de 0 até $2^4 - 1$.

Resumindo, em um processador programável de seis instruções temos as seguintes instruções:

Tabela 1: Conjunto de instruções de seis instruções

Instrução	**Significado**
MOV Ra, d	RF[a] = D[d]
MOV d, Ra	D[d] = RF[a]
ADD Ra, Rb, Rc	RF[a] = RF[b] + RF[c]
MOV Ra, #C	RF[a] = C
SUB Ra, Rb, Rc	RF[a] = RF[b] - RF[c]
JMPZ Ra, offset	PC = PC + offset se RF[a] = 0

Devemos também estender a noção da unidade de controle e bloco operacional para o processador de seis instruções)(figura 12). Primeiramente, a instrução de carregar constante necessita que possamos carregar o valor de $IR[7...0]$ no banco de registradores, além dos valores da memória de dados e da saída da ULA. Assim, aumenta-se o multiplexador do banco de registradores de 2x1 para 3x1, adiciona-se também mais um sinal de controle ao multiplexador e cria-se um sinal novo denominado RF_w_data que vai do bloco de controle se ligando à $IR[7...0]$ e entrano no multiplexador.

Segundo, a instrução subtrair requer que a ULA seja capaz de realizar a subtração. Então, adiciona-se um novo sinal de controle à ULA visando diferenciar quando fazer uma subtração e quando fazer uma adição, alu_s1.

Por fim, a instrução saltar se zero requer a capacidade de detectar se um registrador é zero e somar $IR[7...0]$ ao PC. Assim, insere-se um componente no bloco operacional para determinar se a porta de leitura Rp do banco de registradores es´ta com todos os bits em zero. Também, modifica-se PC para que seja possível carregar $PC = PC + IR[7...0] \therefore PC = PC + offset$. Vale mencionar que o somador utilizado já subtrai 1 da soma, compensando o fato que o estado de Busca já incrementa 1 ao PC.

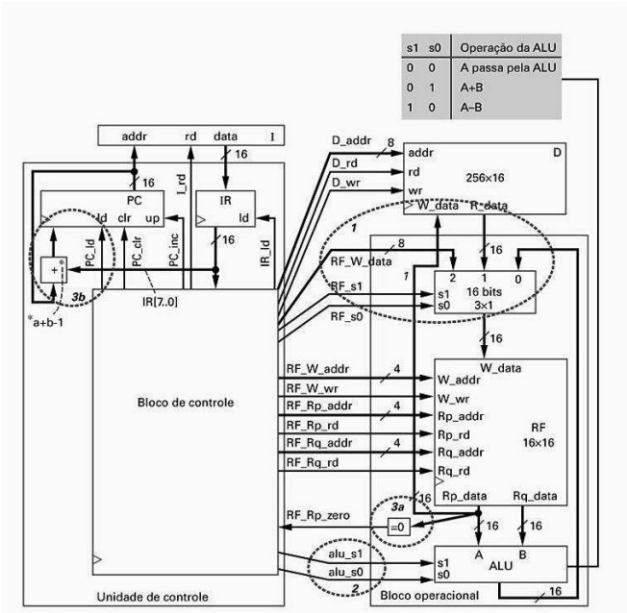

Figura 12: Unidade de controle e bloco operacional e interfaces entra ambos para um processador de seis instruções.

Vamos estender também, naturalmente, a FSM do bloco de controle para lidar com as instruções adicionais. A figura 13 mostra seu diagrama:

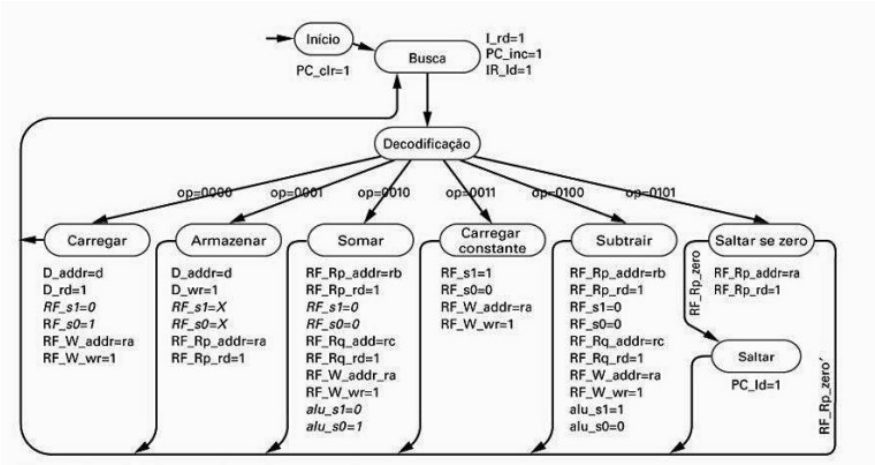

Figura 13: FSM de baixo nível do processador de seis instruções

Os estados início e busca permanecem os mesmos. Em decodificação, adiciona-se três novas transições para as três novas instruções. Também, os estados carregar, armazenar e somar sofreram alterações para refletir a necessidade do multiplexador do banco de registradores possuir duas variáveis de seleção e a ULA ser configurada com dois sinais. Além disso, foram adicionados os três estados extras:

1. No estado Carregar constante, configura-se o multiplexador do banco de registradores para deixar passar o sinal RF_W_data e o banco de registradores para escrever no endereço especificado por ra(IR[11...8]);

2. No estado Subtrair, executa-se as mesmas ações do estado Somar, exceto que a ULA é configurada para subtração ao invés de adição ($s1 = 1$ e ($s0 = 0$);

3. No estado de Saltar se zero, configura-se o banco de registradores para que o registrador especificado por ra seja lido e seu conteúdo colocado na port ade leitura Rp. Se o valor de Rp for composto apenas de 0s, então a saída RF_Rp_zero irá ser 1(e 0, caso contrário). Dessa forma, deve-se tratar de duas transições partindo do estado Saltar se zero. Se $RF_Rp_zero = 0$, ou seja, o registrador lido(presente agora na saída Rp) não contém apenas 0s, a FSM volta para o estado Busca, ou seja, não ocorre salto. Se $RF_Rp_zero = 1$, o registrador lido é composto só de zeros, então a FSM vai para o próximo estado, Saltar, na qual realmente ocorro o salto com o sinal de load do PC sendo 1($PC_ld = 1$).

5 Manipulação de bits e bytes

O Picoblaze realiza atividades de controle e manipulação de dados em baixo nível, realizando a manipulação de dados baseado em bytes. Para manipulação de bits, é necessário compreender as diretivas de baixo nível para controle de I/O, tais como setar, limpar e comparar flags de 1 bit. Devemos estabelecer a convenção utilizada:

1. O símbulo ':' indica um endereço depois de um nome simbólico
2. O símbulo ';' indica um comentário
3. kk indica uma constante, onde k é um dígito hexadecimal

O conjunto de operações de instruções do Picoblaze é orientado para a manipulação de bytes(byte-oriented). Para manipular um único bit é comum o uso de máscaras que permitar extrair, isolar e preservar os bits não relacionados e então aplicar as operações nos bits desejados. As operações mais utilizadas para controle e manipulação de dados são setar, limpar, inverter, extrair e comparar. As três primeiras podem ser feitas com instruções de or, and e xor, respectivamente. Para extrair um bit de um registrador usa-se a isntrução test e a máscara apropriada dependendo do bit desejado.

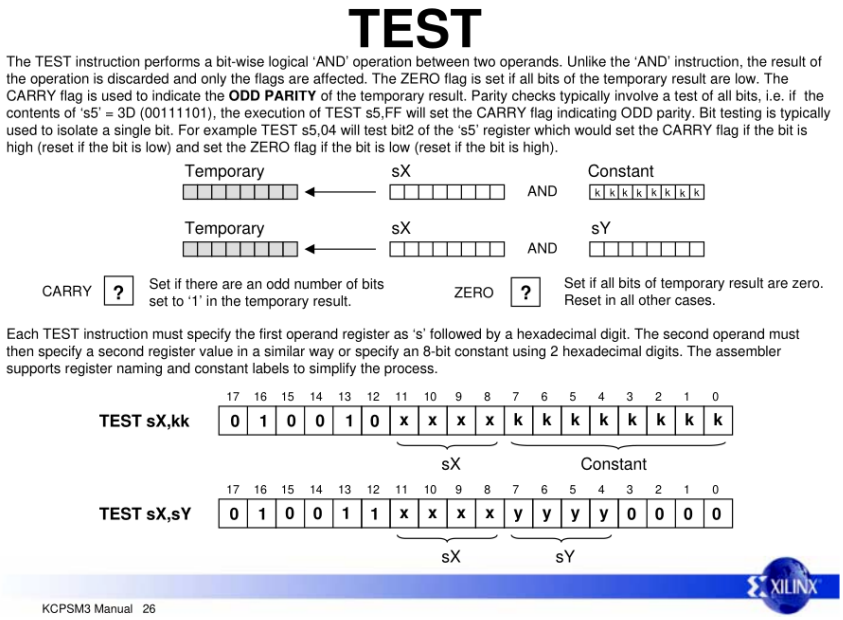

Figura 14: Instrução Test PicoBlaze

5.1 Exemplo 1 - Setar, limpar e inverter o segundo bit LSB do registrador s0

```
constant set_mask, 02 ; mascara=0000_0010
constant clr_mask, FD ; mascara=1111_1101
constant tog_mask, 02 ; mascara=0000_0010

or s0, set_mask       ; xxxx_xxxx
                      ; 0000_0010 or
                      ; xxxx_xx1x

and s0, clr_mask      ; xxxx_xxxx
                      ; 1111_1101 and
                      ; xxxx_xx0x

xor s0, tog_mask      ; xxxx_xxxx
                      ; 0000_0010 xor
                      ; xxxx_xxx'x
```

5.2 Exemplo 2 - Extrair o MSB do registrador s0 e armazenar no registrado s1.

```
constant clr_mask,80 ; mascara=1000_0000

load s1, 00
test s0, clr_mask    ; xxxx_xxxx
                     ; 1000_0000 and
                     ; x000_0000
jump z, done         ; pula para done se z = 1, ou seja, MSB    0
load s1, 01          ; MSB n o    0, portanto, s1=1
```

5.3 Manipulação de múltiplos bytes

Muitos programas precisam manipular dados com múltiplos bytes. Contudo, o microcontrolador Picoblaze é de 8 bits; precisamos, então, de um mecanismo para propagar informação entre duas instruções sucessivas. Para as operações aritméticas, pode-se usar o carry e as instruções de soma e subtração com e sem carry(add, addcy, sub, subcy). Para as operações de deslocamento e rotação, pode-se usar o carry e as instruções de deslocamento ou rotação à direita e à esquerda e à esquerda com ou sem carry(sl0, sla, sr0, sra, rl, rr)

5.3.1 Somando e subtraindo dois números x e y de 24 bits.

```
namereg s0, x0 ; byte LS de x
namereg s1, x1 ; byte do meio de x
namereg s2, x2 ; byte MS de x
;-----------------------------------------------
namereg s3, y0 ; byte LS de y
namereg s4, y1 ; byte do meio de y
namereg s5, y2 ; byte MS de y
;-----------------------------------------------
add    x0, y0
addcy  x1, y1
addcy  x2, y2
```

```
namereg s0, x0 ; byte LS de x
namereg s1, x1 ; byte do meio de x
namereg s2, x2 ; byte MS de x
;-----------------------------------------------
namereg s3, y0 ; byte LS de y
namereg s4, y1 ; byte do meio de y
namereg s5, y2 ; byte MS de y
;-----------------------------------------------
sub    x0, y0
subcy  x1, y1
subcy  x2, y2
```

5.3.2 Incrementar uma palavra x de 24 bits que ocupa três registradores

```
namereg s0, x0
namereg s1, x1
namereg s2, x2
add x0, 01
addcy x0, 00
addcy x0, 00
```

5.3.3 Deslocar uma posição para esquerda de uma palavra de 24 bits

```
namereg s0, x0
namereg s1, x1
namereg s2, x2
sl0 x0 ; desloca com 0 no LSB de X, MSB de x vai para o carry
sla x1 ; desloca    esquerda com o carry da instru   o anterior
sla x2 ; desloca    esquerda com o carry da instru   o anterior
```

5.3.4 Somar D[5]=D[5]+D[6]+D[7]

```
FETCH s1, 05
FETCH s2, 06
FETCH s3, 07
LOAD s0, 00
ADD s0, s1
ADD s0, s2
ADD s0, s3
STORE s0, 05
;-------------------------USING CONSTANT
CONSTANT D5, 05
CONSTANT D6, 06
CONSTANT D7, 07
main: FETCH s1, D5
FETCH s2, D6
FETCH s3, D7
LOAD s0, 00
ADD s0, s1
ADD s0, s2
ADD s0, s3
STORE s0, D5
```

5.3.5 Somar D[9]=D[0]+D[1]

```
CONSTANT D9, 09
CONSTANT D0, 00
CONSTANT D1, 01
main:
FETCH s0, D0
FETCH s1, D1
ADD s0, s1
STORE s0, D9
```

6 Estruturas de controle de programa com Picoblaze

Linguagens de programação de alto nível como C e C++ contém diversas estruturas de controle para alterar a execução de um programa. Tipicamente, o controle de fluxod e programa é feito pelas diretivas:

- if-then-else
- case
- for-loop

O Picoblaze apenas oferece instruções jump condicionadas e não-condicionados. Assim sendo, as instruções jump podem ser usadas junto com instruções test e/ou compare para implementar as estruturas de controle de fluxo de alto nível.

6.1 Estutura if-then-else

A estrutura if(s0==s1) pode ser implementada usando **compare** e **jumpnz**

```
if(s0==s1){
  /* then-branch statements */
}else{
  /* else-branch statements*/
}
```

Lembrando da instrução **compare**:

Flag States		Comparison
Z	C	
0	0	sX > kk or sX > sY
x	1	sX < kk or sX < sY
1	x	sX = kk or sX = sY

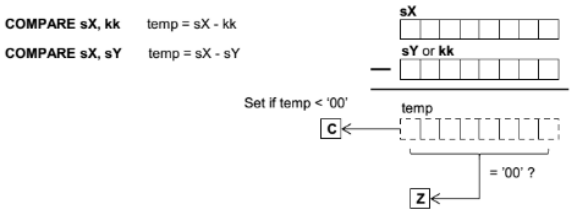

Figura 15: Instrução compare Picoblaze e estados das flags

Table C-1: **CALL Instruction Conditions**

Condition	Description
<none>	Always true. Call subroutine unconditionally.
C	CARRY = 1. Call subroutine if CARRY flag is set.
NC	CARRY = 0. Call subroutine if CARRY flag is cleared.
Z	ZERO = 1. Call subroutine if ZERO flag is set.
NZ	ZERO = 0. Call subroutine if ZERO flag is cleared.

Figura 16: CALL Instruction Conditions

```
if(s0==s1){
  /* then-branch statements */
}else{
  /* else-branch statement*/
}
```

```
compare s0,s1;if compare yields 0(s0-s1==0)
jump nz, else_branch;will only execute if
  ;code for if-then branch
jump if_done
else_branch:
  ;code for else branch
if_done:
  ;code following if-then-else statement
```

6.2 Estrutura case

A estrutura **case** pode ser vista como múltiplos saltos de instrução. Como já sabemos implementar if-then-

```
  switch(s0){
    case value1:
      /*case value 1 statements*/
      break;
  }
    case value2:
      /*case value 2 statements*/
      break;
  }
    case value3:
      /*case value 3 statements*/
      break;
  }
    default:
      /* default statements*/
}
```

```
if(s0==value1){
  /*case value1 statements*/
}
else if(s0==value2){
  /*case value2 statements*/
}
else if(s0==value3){
  /*case value3 statements*/
}
else{
  /*default statements*/
}
```

else, a tarefa é similar:

```
if(s0==value1){
  /*case value1 statements*/
}
else if(s0==value2){
  /*case value2 statements*/
}
else if(s0==value3){
  /*case value3 statements*/
}
else{
  /*default statements*/
}
```

```
        constant value1, ...
        constant value2, ...
        constant value3, ..

        compare s0, value1 ;test value1
        jump nz, case_2
;not equal to value 1, jump to case2
        ;code for case 1
        ...
        jump case_done

      case_2:
        compare s0, value2
        jump nz, case_3
        ;code for case 2
        ...
        jump case_done

      case_3:
        compare s0, value3
        jump default
        ;code for case 3
        ...
        jump case_done

      default:
        ;code for default case
        ...
      case_done:
        ;code following case statement
```

6.3 Estrutura for-loop

A estrutura **for** executa uma sequência de código repetidamente. Esta estrutura pode ser implementada usando um contador de iterações.

```
for(i=MAX; i=0; i=i-1){
  /*loop body statements*/
}
```

```
  namereg s0, i       ;loop index
  constant MAX, ...   ;loop boundary
  load i, MAX ;load loop index

loop_body:
  ;code for loop body
  ...
  sub i, 01 ;decrease loop index
  jump nz,loop_body ;done?

  ;code following for loop
  ...
```

7 Subrotinas

Subrotinas em linguagens de programação de alto nível são porções de um programa maior. Geralmente, implementam tarefas específicas usadas repetitivamente e permitem, assim, dividir o programa em partes menores e mais manejáveis. O Picoblaze utiliza as intruções **call** e **return** para implementar subrotinas. A instrução **call** salva o conteúdo atual do contador do programa e transfere a execução do programa para o endereço inicial da subrotina. A subrotina termina com a instrução **return**, que recupera o valor do contador de programa e retorna a execução do programa. **Nota:** Durante uma chamada de função, o Picoblaze apenas armazena e restaura o conteúdo do contador de programa. Assim, não é preservado o contexto do banco de registradores nem das flags. O projetista deve sempre cuidar manualmente do bankreg e RAM de forma que o estado do sistema não seja alterado após a chamada da subrotina.

A instrução **call** salva o valor do contador do programa na **pilha**, incrementa o **tos** e transfere a execução do programa para o endereço inicial da subrotina. A instrução **return** recupera o valor do contador de programa e retoma a execução do programa. Por termos uma pilha de 31 posições, é possível ter até 31 subrotinas aninhadas.

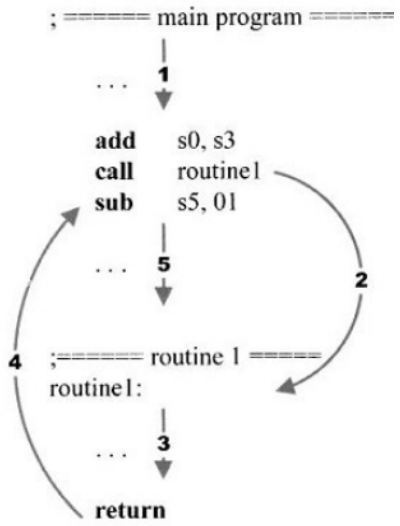

Figura 17: Funcionamento base da chamada e retorno de uma subrotina